CHAPTER 1

SYSTEM DESIGN INTERVIEWS

1 Getting Ready for the System Design Interview 2
2 The Do's and Don'ts of the System Design Interview 6
3 How Long Does It Take to Prepare for a System Design Interview? . . . 8
4 System Design Mock Interviews . . . 12
5 Let AI Evaluate Your System Design Interview Preparation 17

Explore what System Design Interviews involve, including essential preparation strategies, fundamental concepts, key resources, and tips to perform well.

1 Getting Ready for the System Design Interview

A **System Design interview** is a technical evaluation of a candidate's ability to build robust and scalable systems. Unlike coding interviews, which typically involve a single solution, System Design interviews are open to discussion and involve multiple possible solutions that can be re-iterated.
For many software engineers, System Design Interview questions remain a mysterious challenge. Most engineers have never actually worked on large-scale systems before, so explaining how to build one seems daunting. Since System Design interview questions are often open-ended, it can be difficult to know the best way to prepare.
However, as someone who has participated in hundreds of System Design interviews, I can assure you there is a correct way to approach these questions. With the right mindset and preparation, you can feel confident and ready to tackle any System Design question that comes your way.
It is important to note that System Design questions not only test the technical knowledge of the candidate but also their ability to approach a problem, think critically, and make trade-offs. Therefore, preparing for a System Design interview is not only about understanding the technical details but also about understanding the problem, breaking it down, and finding the most optimal solution.

1.1 Author's background in System Design

Hi, I'm Fahim ul Haq, the co-founder of Educative and co-author of this course. In April 2008, I joined an internal team at Microsoft working on a large-scale project to build a distributed storage solution. Amazon launched its Simple Storage Service in 2006, and Google launched its PaaS solution, Google App Engine, the same month I joined the team, so we were in the early land grab of cloud computing. Less than two years later, that project was launched to the world as a new product category: Microsoft Azure.
When I joined the Azure team, I came from working on Exchange. I understood server storage and client management, but not at this scale and certainly not distributed across the world. It required a lot of learning on the job.
Today, the lessons I and other cloud engineers learned in those early days are codified into the System Design discipline. For many companies, the System Design Interview is instrumental in the developer interview process, which means it is vital for landing a job and setting your career on a good trajectory.

What I learned through hundreds of System Design Interviews

By the time I started Educative, I had participated in hundreds of interview loops as both interviewee and interviewer. As Educative has scaled, I have participated in hundreds more. The experience of working on web-scale systems at Facebook and Microsoft taught me two key skills for approaching the System Design Interview:

1. How to learn the fundamentals of distributed systems quickly and apply these principles in solving real-world problems
2. How to evaluate candidates while interviewing for System Design

Here's the counterintuitive part: in the System Design Interview, companies are not actually trying to test your experience with System Design. Successful candidates rarely have much experience working on large-scale systems, and interviewers know this. Again , this discipline has only been around for about fifteen years, and like everything else in software engineering, it is evolving rapidly.

> **Note:** The key is to prepare for the System Design interview with the intent to apply that knowledge.

Our System Design course is equally useful for people already working and those preparing for interviews. In this chapter, we will highlight the different aspects of a System Design interview and some helpful tips for those who are preparing for an upcoming interview. We encourage learners to read this chapter even if they aren't preparing for an interview because some of the topics covered in this chapter can be applied broadly.

1.2 How do System Design interviews differ from other interviews?

Just like with any other interview, we need to approach the System Design interviews strategically. System Design interviews are different from coding interviews. There's rarely any coding required in this interview.

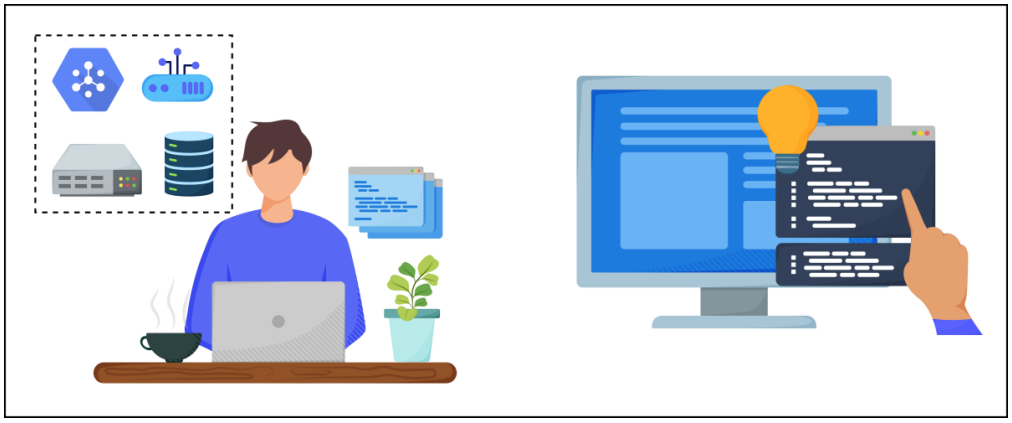


Figure 1.1: Other interviews versus a system design interviews

A System Design interview takes place at a much higher level of abstraction. We figure out the requirements and map them onto the computational components and the high-level communication protocols that connect these subsystems.
The final answer doesn't matter. What matters is the process and the journey that a good applicant takes the interviewer through.

> **Note:** As compared to coding problems in interviews, System Design is more aligned with the tasks we'll complete on our jobs.

1.3 How do we tackle a design question?

Design questions are open ended, and they're intentionally vague to start with. Such vagueness mimics the reality of modern day business.
Interviewers often ask about a well-known problem—for example, designing WhatsApp. Now , a real WhatsApp application has numerous features, and including all of them as requirements for our WhatsApp clone might not be a wise idea due to the following reasons:

- First, we'll have limited time during the interview.
- Second, working with some core functionalities of the system should be enough to exhibit our problem-solving skills.

We can tell the interviewer that there are many other things that a real WhatsApp does that we don't intend to include in our design. If the interviewer has any objections, we can change our plan of action accordingly. Here are some best practices that we should follow during a System Design interview:

- An applicant should ask the right questions to solidify the requirements.
- Applicants also need to scope the problem so that they're able to make a good attempt at solving it within the limited time frame of the interview. System Design interviews are usually about 35 to 40 minutes long.

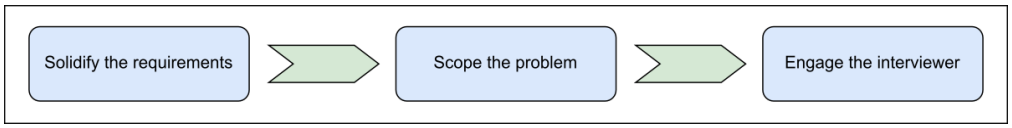


Figure 1.2: Best practices for the System Design interview

- Communication with the interviewer is critical. It's not a good idea to silently work on the design. Instead , we should engage with the interviewer to ensure that they understand our thought process.

Present the high-level design

At a high level, components could be frontend, load balancers, caches, data processing, and so on. The System Design explains how these components fit together.
An architectural design often represents components as boxes. The arrows between these boxes represent who talks to whom and how the boxes or components fit together collectively.

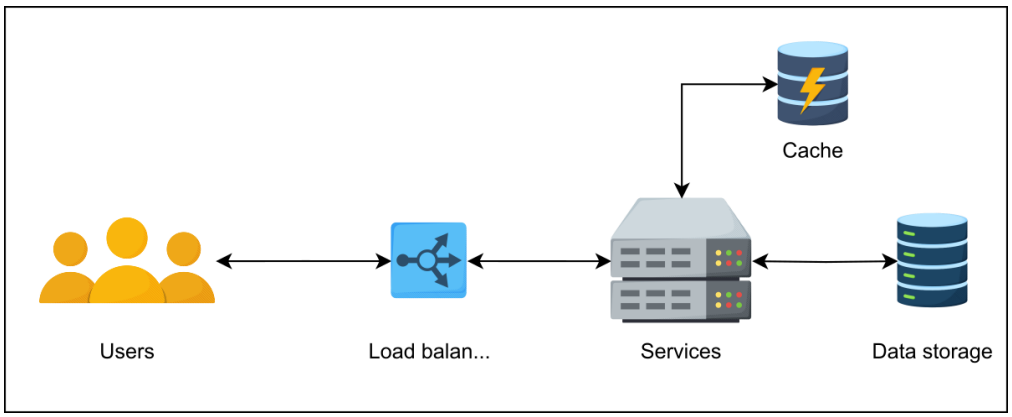


Figure 1.3: A sample design

1.4 Possible questions for every System Design interview

System Design interviews often include questions related to how a design might evolve over time as some aspect of the system increases by some order of magnitude—for example, the number of users, the number of queries per second, and so on. It 's commonly believed in the systems community that when some aspect of the system increases by a factor of ten or more, the same design might not hold and might require change. Designing and operating a bigger system requires careful thinking because designs often don't linearly scale with increasing demands on the system.
A common question in a System Design interview is why we don't design a system that can handle more work than necessary or predicted.

Try answering this yourself using the space below.
Question:
Evolving System Design
Reference Answer:
Overbuilding a system upfront leads to wasted resources and added complexity. Good design focuses on meeting current needs efficiently while allowing for easy scaling as demand grows. This keeps systems simpler, cheaper, and easier to manage.

Design challenges

Things will change, and things will break over time because of the following:

- There's no single correct approach or solution to a design problem.
- A lot is predicated on the assumptions we make.

The responsibility of the designer

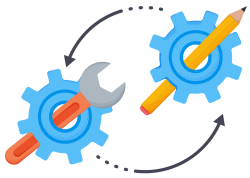
As designers, we need to provide fault tolerance at the design level because almost all modern systems use off-the-shelf components, and there are millions of such components. So , something will always be breaking down, and we need to hide this undesirable reality from our customers.

1.5 Who is eligible for a System Design interview?

Traditionally, mid-to-senior level candidates with more than two years of experience get at least one System Design interview. For more senior applicants, two or three System Design interviews are common. Recently , large companies have also put forth System Design questions to some junior candidates. It 's never too early to learn System Design to grow or even expedite our careers.

1.6 Theory and practice

Most of the theory of System Design comes from the domain of distributed systems. Getting a refresher on these concepts is highly recommended. Educative has an excellent course on distributed systems that we can use to refresh our knowledge of distributed systems concepts.



Distributed systems give us guideposts for mature software principles. These include the following:

- Robustness (the ability to maintain operations during a crisis)
- Scalability
- Availability
- Performance
- Extensibility
- Resiliency (the ability to return to normal operations over an acceptable period of time post-disruption)

Such terminology also acts as a lingua franca between the interviewer and the candidate.
As an example, we might say that we need to make a trade-off between availability and consistency when network components fail because the CAP theorem indicates that we can't have both under-network partitions. Such common language helps with communication and shows that we're well-versed in both theory and practice.

When does the System Design interview occur in the hiring process?

In FAANG+ (Facebook, Apple, Amazon, Netflix, Google, Microsoft, etc.), the hiring process involves several technical and non-technical interview rounds. The System Design interview usually comes after coding interview rounds and before behavioral interviews. A typical occurrence of a System Design interview in the hiring process is depicted below:

> **Remember:** In a System Design interview, It's a candidate's job to exhibit their skills to the interviewer.

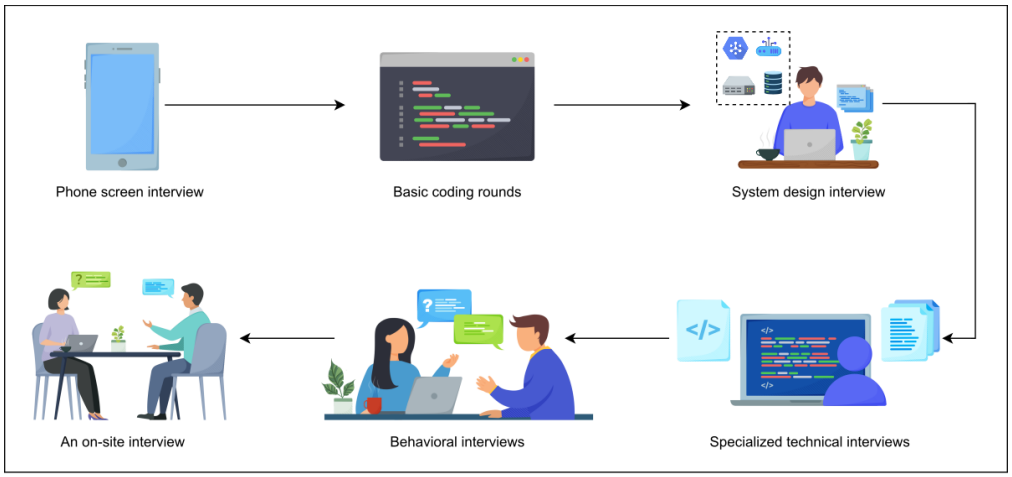


Figure 1.4: System Design interview, among other technical and nontechnical interviews

2 The Do's and Don'ts of the System Design Interview

2.1 What to do during the interview

We stress that a candidate should make an effort to avoid looking unoriginal. The interviewer has probably asked the same question to many candidates. Reproducing a run-of-the-mill design might not impress the interviewer.
At the same time, an interview can be a stressful situation. Having a plan to attack the problem might be a good strategy. Depending on the candidate, there can be multiple strategies to attack a design problem. We suggest the following technique.

2.2 Strategize, then divide and conquer

We recommend including the following activities somewhere in the interview:

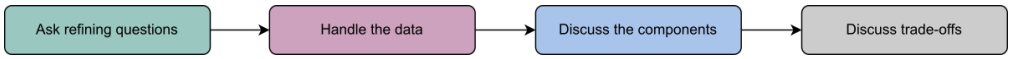


Figure 1.5: Activities to include in the interview

Ask refining questions

We need to understand the design problem and its requirements. To do this, we can put on our product manager hat and prioritize the main features by asking the interviewer refining questions. The idea is to go on a journey with the interviewer about why our design is good. These interviews are designed to gauge if we're able to logically derive a system out of vague requirements.
We should ensure that we're solving the right problem. Often, it helps to divide the requirements into two groups:

- Requirements that the clients need directly—for example, the ability to send messages in near real-time to friends.
- Requirements that are needed indirectly—for example, messaging service performance shouldn't degrade with increasing user load.

  Note: Professionals call these functional and nonfunctional requirements.

Handle data

We need to identify and understand data and its characteristics in order to look for appropriate data storage systems and data processing components for the system design.
Some important questions to ask ourselves when searching for the right systems and components include the following:

- What's the size of the data right now?
- At what rate is the data expected to grow over time?
- How will the data be consumed by other subsystems or end users?
- Is the data read-heavy or write-heavy?
- Do we need strict consistency of data, or will eventual consistency work?
- What's the durability target of the data?
- What privacy and regulatory requirements do we require for storing or transmitting user data?

Discuss the components

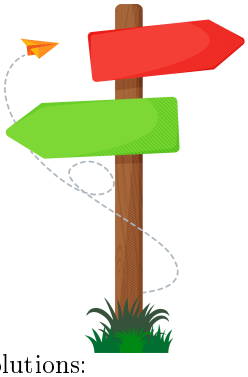
At some level, our job might be perceived as figuring out which components we'll use, where they'll be placed, and how they'll interact with each other.
An example could be the type of database—will a conventional database work, or should we use a NoSQL database?

There might be cases where we have strong arguments to use NoSQL databases, but our interviewer may insist that we use a traditional database. What should we do in such a case?
As designers, we'd have a harder job because we'd need to use a traditional database and do extra work to ameliorate the shortcomings or challenges. In this case, we'd have invented a new component.
Such interactions during interviews are also excellent opportunities to exhibit our design skills.

> **Note:** We often abstract away the details of the components as boxes and use arrows to show the interactions between them. It might help to define the user-facing APIs at a high level to further understand system data and interaction requirements.

Front-end components, load balancers, caches, databases, firewalls, and CDNs are just some examples of system components.

Discuss trade-offs
Remember that there's no one correct answer to a design problem. If we give the same problem to two different groups, they might come up with different designs.



These are some of the reasons why such diversity exists in design solutions:

- Different components have different pros and cons. We'll need to carefully weigh what works for us.
- Different choices have different costs in terms of money and technical complexity. We need to efficiently utilize our resources.
- Every design has its weaknesses. As designers, we should be aware of all of them, and we should have a follow-up plan to tackle them.

We should point out weaknesses in our design to our interviewer and explain why we haven't tackled them yet. An example could be that our current design can't handle ten times more load, but we don't expect our system to reach that level anytime soon. We have a monitoring system to keep a very close eye on load growth over time so that a new design can be implemented in time. This is an example where we intentionally had a weakness to reduce system cost.
Something is always failing in a big system. We need to integrate fault tolerance and security into our design.

2.3 What not to do in an interview

Here are a few things that we should avoid doing in a system design interview:

- Don't write code in a system design interview.
- Don't start building without a plan.
- Don't work in silence.
- Don't describe numbers without reason. We have to frame it.
- If we don't know something, we don't paper over it, and we don't pretend to know it.

  Note: If an interviewer asks us to design a system we haven't heard of, we should just be honest and tell them so. The interviewer will either explain it to us or they might change the question.

3 How Long Does It Take to Prepare for a System Design Interview?

System Design interviews assess a candidate's ability to design large-scale distributed systems.
As technologies such as the internet and artificial intelligence increasingly rely on distributed architectures,

most companies expect applicants to understand the tidbits of System Design. Preparation for the System Design interview should make up a significant part of a technical interview study plan.
This lesson outlines how to plan an effective preparation timeline and allocate time appropriately.

3.1 The format of a System Design interview

System Design interviews differ from coding interviews in that there can be more than one correct answer to a problem.
Interviewers are often more interested in the candidate's thought process than the final design. A successful System Design interview depends on the ability to communicate reasoning clearly and hold a structured discussion.
Interviewees typically have access to a whiteboard (in-person or virtual) to illustrate their approach. After introductions and ice-breakers, a typical interview format includes the following stages:

- Open-ended problem statement (e.g., How would you design Instagram?)
- Requirement gathering (clarifying details such as load, response times, scalability, availability, fault tolerance, and security).
- Design discussion, progressing from high-level to detailed components.
- Evaluation and feedback, where design trade-offs are justified.

  Note: Mastering technical fundamentals is essential for System Design interview preparation. However, holistic preparation should account for both technical concepts and soft skills that demonstrate clear communication and collaboration.

3.2 What to prepare for

Effective System Design interview preparation should address both technical and interpersonal aspects:

- Distributed systems concepts and design patterns
- System components such as load balancers, caches, and databases
- Practice with common System Design interview questions, such as Designing a YouTube system, Designing a URL Shortener, and Designing a Twitter system.
- Mock interviews to build fluency and confidence
- Company research, including technology stacks and engineering culture

  Tip: Like behavioral interviews, System Design rounds also assess culture fit. Studying a company's values alongside its technical practices provides valuable insight. Many large organizations such as Amazon, Microsoft, Apple, and Netflix openly share their cultural principles.

3.3 How long to prepare

Assuming a commitment of 2–3 hours per day for five days a week, preparation timelines typically fall into the following ranges:

- 4–10 weeks for candidates starting from scratch or requiring additional study time
- 3–6 weeks for those with prior experience in System Design

An 8-week example plan is illustrated below:

Preparation duration can vary significantly based on individual factors. Some candidates may be eligible to extend their study period by up to 16 weeks, depending on their prior experience, pace, and availability.
The total preparation time generally depends on three major factors:

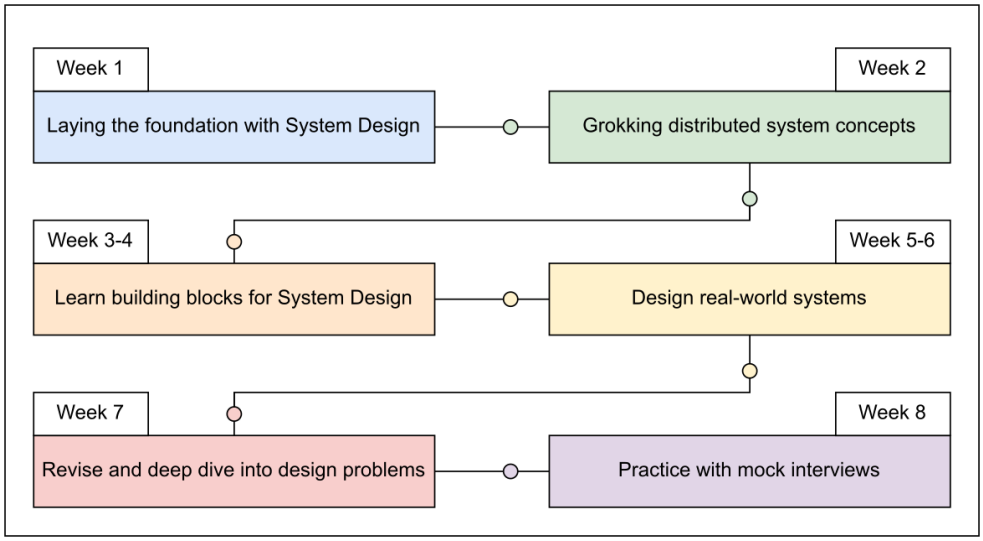


Figure 1.6: A roadmap for System Design interview preparation

1. Individual circumstances
2. Current skill level
3. Role or level being pursued

Individual circumstances

Non-technical factors often influence the time required for preparation. These include:

- Learning speed
- Available time for study

When System Design concepts are new, additional time should be allocated for understanding and internalization. Candidates balancing full schedules may require extended preparation timelines.

> **Note:** The objective is internalization, not memorization. The interview is not a memory test. Candidates must apply learned concepts to novel problems, reason about trade-offs, and justify design decisions under given constraints.

Skill level

Preparation time also depends on prior experience and familiarity with distributed systems:

- **Prior experience:** Those who have already designed or worked with large-scale systems may require less preparation.
- **Tech stack familiarity:** Candidates new to specific technologies should budget extra time to understand their architecture and constraints.

True mastery takes time, particularly when working with complex topics such as scalability, fault tolerance, and data consistency. A slower, steady pace allows concepts to settle and promotes long-term retention.
Lack of experience with a company's specific tech stack does not disqualify a candidate. Many employers focus on design reasoning and adaptability rather than specialized tool knowledge. The primary goal should be to strengthen fundamental System Design skills.

The role or level being pursued

The depth of knowledge required varies by position. The following table outlines typical expectations for each role or level:

Role/Level	Key Focus Areas
Junior/Mid-level Software Engineer	Basic System Design components; Trade-offs (e.g., between requirements such as latency and availability)
Senior Software Engineer	Explain how system components interact; describe the complete life cycle of a request (from client to server to backend); align trade-offs with user and product goals.
Staff Software Engineer	Identify potential bottlenecks; anticipate scaling issues and possible solutions; describe failover and recovery mechanisms; assess how failures affect user experience and SLAs.
Principal/Distinguished Engineer	Design for regional backups and disaster recovery; analyze query and usage patterns; evaluate how outages or security breaches affect company reputation and SLA commitments.
Technical Product or Project Manager	Understand scalable system behavior at a conceptual level; recognize distributed system components; assess how budget and stakeholder constraints influence technology decisions.

> **Note:** Higher levels build upon earlier ones. Advanced engineers are expected to demonstrate mastery of all topics covered at preceding levels.

Depending on these factors, some candidates may need to allocate additional time to fully prepare. An extended study may be especially beneficial for those balancing professional responsibilities or approaching System Design for the first time.

3.4 Conclusion

System Design interviews remain a critical part of technical hiring processes. Effective preparation requires a balance of consistent study, conceptual understanding, and communication practice. Developing this skill set is crucial for long-term career growth in the technology sector.

Estimated preparation timelines include:

- 3–6 weeks for those with prior experience
- 4–10 weeks for beginners
- Up to 16 weeks for extended preparation

System Design mastery develops over time. Preparation should emphasize understanding rather than memorization.

Candidates who communicate clearly, articulate trade-offs, and demonstrate collaborative thinking will leave a lasting impression. Practicing System Design problems alone is not sufficient. Soft skills and reasoning clarity are equally crucial for success.

In the next lesson, we'll discuss how mock interviews can help develop and sharpen both technical and soft skills.

4 System Design Mock Interviews

Even experienced software and infrastructure engineers can feel pressure when facing a System Design interview.
It is not just the open-ended questions or the high expectations from companies such as Facebook (Meta), Amazon, Apple, Netflix, and Google (FAANG or MAANG). The core challenge lies in translating complex real-world requirements into efficient, large-scale distributed systems within tight time limits.
System Design interviews test a candidate's technical skills and ability to think independently while balancing trade-offs.
Knowing the theory is not enough. A deep understanding of real-world distributed systems is essential for solving complex problems in real-time. Given these challenges, a practical approach to preparation is to engage in mock interviews. These provide a hands-on way to refine strategies and problem-solving skills.

4.1 The role of mock interviews

Mock interviews for System Design are one of the best ways to prepare for actual interviews.
They recreate the pressure and environment of a real interview, giving candidates a chance to experience the intensity firsthand. From presenting an initial solution to defending it during a discussion, mock interviews encourage candidates to think quickly and handle unexpected questions.
Candidates should be prepared before attending mock interviews.
To get the most out of the experience, it is important to study key concepts and practice different design problems in advance. A mock interview begins with a design problem. This is followed by a discussion phase in which the candidate scopes the problem and explains design choices within realistic time constraints.
After questions covering both high-level design and detailed components, the interviewer provides feedback about performance, including strengths and areas for improvement.
Mock interviews build confidence, strengthen communication skills, and improve the ability to reason through trade-offs.

Benefits of mock interviews

Mock interviews offer several advantages for preparation:

- **Practice under pressure:** They allow candidates to practice under time limits and handle high-stress scenarios in a low-risk setting.
- **Feedback from the interviewer:** Feedback helps identify weaknesses and target specific areas for improvement.
- **Building confidence:** Repeated practice develops confidence and makes real interviews easier to manage.
- **Exposure to different scenarios:** Candidates experience different questioning styles and system types, which improves adaptability and problem-solving ability.

4.2 The AI mock interviewer

To replicate the experience of real System Design interviews, *Educative* created AI-driven mock interviewers using generative AI.
These allow candidates to engage in simulated interviews, practice articulating their thoughts, and refine problem-solving approaches in an environment similar to a real interview. Although AI mock interviews cannot fully replace human interviewers, they provide several important advantages.

- **Accessibility:** They are available anytime and anywhere, allowing flexible preparation.
- **Instant and unbiased feedback:** They provide real-time evaluations that are objective and unbiased.

- **Cost-effective:** They are less expensive than traditional mock interviews and allow for more frequent practice.
- **Adaptability:** They adjust the difficulty of questions based on previous performance and experience level.
- **Constantly improving:** They are updated with new insights, recruitment trends, and user feedback.
- **Developed by FAANG experts:** They are built with input from experienced engineers to ensure realistic content.
- **Integration with learning:** They connect with System Design courses to create a unified learning and practice cycle.

Educative's mock interviews for System Design

Educative provides AI-powered interviewers across several domains.
In addition to company-specific interviews designed to help candidates prepare for roles at organizations such as Meta, Amazon, Apple, Netflix, Google, and Microsoft, the platform offers interviews for many individual design problems.
The given list shows examples from the System Design domain:

- TikTok System Design
- Distributed Cache System Design
- Pub-Sub System Design
- Web Crawler System Design
- Uber Eats System Design
- Zoom System Design
- YouTube System Design
- X (Twitter) System Design
- WhatsApp System Design
- Uber System Design
- Typeahead System Design
- TinyURL System Design
- Ticketmaster System Design
- Spotify System Design
- Reddit System Design
- Payment System Design
- NewsFeed System Design
- Netflix System Design
- Linkedin System Design
- LeetCode System Design
- Instagram System Design
- Google Maps System Design
- Google Docs System Design
- Facebook Messenger System Design
- E-Commerce Store System Design
- Discord System Design
- Deployment System Design
- Content Delivery Network (CDN) System Design
- ChatGPT System Design
- Blob Store System Design
- Apple App Store System Design
- Airbnb System Design

The following illustration shows examples of System Design interviewers on *Educative's* platform:

> **Note:** Mock interviews are also available for other domains such as API design, coding, Generative AI, machine learning, and low-level design.

Steps involved while attempting a mock interview

A mock interviewer follows a structured process to simulate a real System Design interview.

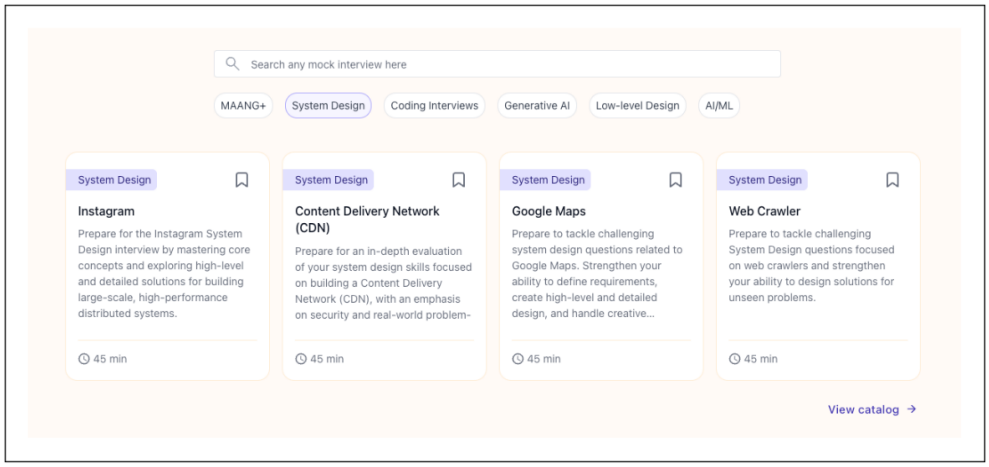


Figure 1.7: Educative's System Design mock interviews

1. **Clarifying requirements:** The interviewer begins by identifying functional and non-functional requirements.

2. **High-level design:** The candidate outlines the overall architecture and explains the interactions between components.

3. **API and data model:** The interviewer asks about the API design and data structures to assess understanding.

4. **Workflow and detailed design:** The candidate explains the system's workflow and reasoning behind design decisions.

The difficulty of questions increases as the interview progresses. Some sample questions asked by the interviewer:

- How would you design the upload process to support high volumes of concurrent video uploads?

- How would you design a system to recommend personalized video content to users? What data would you collect, and how would you process it in real time to update recommendations?

- How would you ensure that the system scales to handle millions of concurrent users while maintaining fault tolerance?

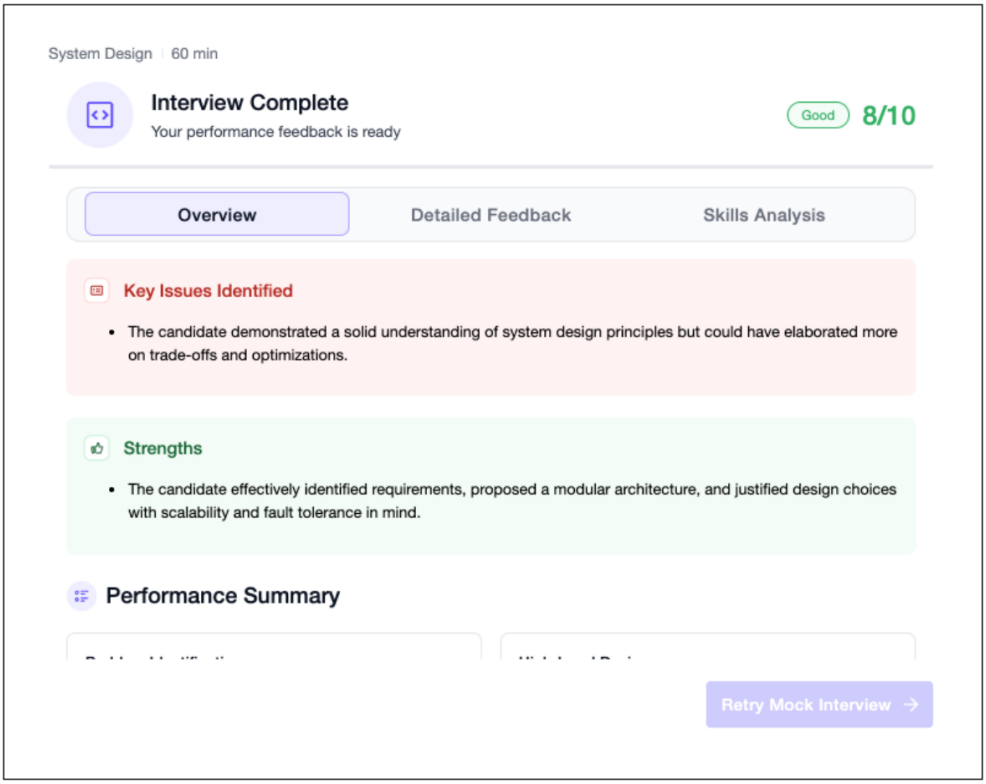
- Let's discuss how you would handle age-restricted content on the YouTube platform. What mechanisms would you put in place to ensure that such content is only accessible to the appropriate audience?

5. **Balancing trade-offs:** The interviewer evaluates the ability to weigh trade-offs such as scalability and performance or consistency and availability.

6. **Feedback:** The interviewer provides a qualitative evaluation rated as Unsatisfactory, Satisfactory, Good, or Excellent.

The following illustration shows a good and average performance provided by the mock interviewer:

Note: The interview lasts about 45 minutes and includes questions of different difficulty levels. The feedback helps candidates address weaknesses and prepare for future interviews.



How to make the most of AI mock interviews

To maximize the benefits of AI mock interviews, candidates should consider the following points:

- **Treat each session as real:** Approach every mock interview with the same focus and preparation as a real one.
- **Focus on feedback:** Review feedback carefully to identify recurring issues and opportunities for improvement.
- **Repeat and refine:** Conduct regular practice sessions to build fluency and confidence.
- **Utilize integrated platforms:** Combine mock interviews with related courses and resources to facilitate continuous learning.

An effective strategy for mock interviews

Beyond taking mock interviews, candidates should strengthen their understanding of core distributed system components:

- DNS
- Load balancers
- Databases
- Key-value stores
- Content Delivery Networks (CDNs)
- Sequencers
- Service monitoring
- Distributed logging systems
- Distributed caches
- Messaging queues
- Pub/sub systems
- Blob storage
- Distributed search
- Task schedulers
- Sharded counters
- Rate limiters

Familiarity with these elements helps candidates design scalable and fault-tolerant systems. Candidates can further improve preparation by categorizing systems by functionality.

1. Video streaming systems such as YouTube and Netflix
2. Real-time communication systems such as WhatsApp and Facebook Messenger
3. Ride-hailing systems such as Uber and Lyft
4. Feed-based social networks such as Instagram and TikTok
5. Cloud-based collaboration or file systems such as Dropbox and Google Drive

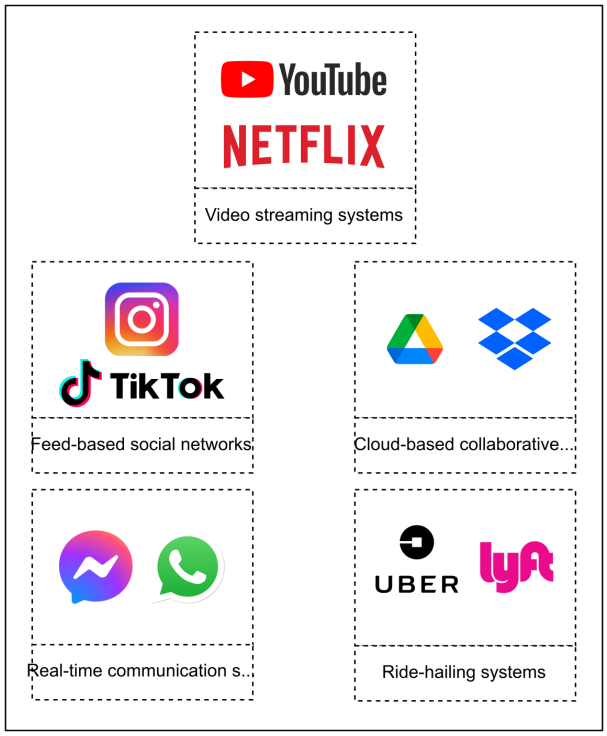


Figure 1.8: Categorization of System Designs based on their functionalities

By understanding these categories, candidates can tailor practice sessions to focus on relevant design challenges.

Common pitfalls to avoid in AI mock interviews

After building a strong preparation plan and utilizing AI mock interviews effectively, candidates should remain mindful of common mistakes that can hinder their improvement. Being aware of these pitfalls ensures steady progress and readiness for real interviews.

- **Ignoring feedback:** Overlooking the feedback after each interview limits progress. Reviewing and acting on insights is essential for growth.
- **Not simulating real conditions:** Skipping realistic conditions, such as time limits or focused environments, reduces the effectiveness of practice. Recreating real conditions ensures better readiness for actual interviews.

4.3 Conclusion

Mastering System Design interviews is critical for securing roles at top tech companies.
Mock interviews, particularly those enhanced by AI, offer a valuable way to develop skills, receive feedback, and simulate the intensity of real interviews. By recreating real interview conditions, providing detailed feedback, and integrating with learning resources, Educative's AI mock interviewers help developers improve their design reasoning and perform confidently in System Design interviews.

5 Let AI Evaluate Your System Design Interview Preparation

5.1 Identifying system requirements

You are in a System Design interview. The interviewer presents an open-ended question, such as: "Design a video streaming service like YouTube."
Your task is to identify:

1. Two functional requirements (what the system does) and
2. Two non-functional requirements (how the system performs).

Question:
Identifying System Requirements
Reference Answer:

1. Requirements:

- functional requirements:
  - Accept any 2 features that describe what a user does on the platform.
  - Examples of valid answers: Uploading videos, streaming/watching content, searching for titles, liking/commenting, user authentication (login/signup).

2. Non-functional requirements:

  - Accept any 2 quality attributes related to system performance.
  - Examples of valid answers: Scalability (handling high traffic), Availability (system uptime), Low Latency (fast loading), Reliability (no data loss), Consistency.

5.2 Scoping the problem

After identifying requirements for a video streaming service in the previous step, your next task is to ask two clarifying questions from the interviewer to scope the problem (specifically regarding data characteristics or user load).

Question:

Scoping the Video Streaming Design Problem

Reference Answer:

1. Refining questions: Accept any 2 questions that help clarify the scope, scale, or constraints of the system.

    - Examples of valid questions: – How many daily active users (DAU) do we have? – What is the average video file size? – Are we targeting a global audience or a specific region? – Do we need to support live streaming?